swotvac-saviour - Plan v2 (folder workflow)

Planning document. Describes the new primary workflow and maps every existing component as reused, adapted, or retired. No implementation in this phase.

Overview

A student opens an omp session in a study directory. The directory contains one subfolder per subject; each subfolder holds the exercise PDFs for that subject. The tool ingests those folders, transcribes the questions with a cheap model, and serves a local site that shuffles the transcribed questions, saves progress, and reports accuracy.

The site the student uses is produced by a template/tool: the LLM inserts the question data into a scaffold, and the tool supplies everything needed to build and run a self-hosted site on top of the database.

Workflow

1. Study layout (student's files, untouched by hand):

```
study-dir/
  linear-algebra/        <- subject folder, becomes a tag
    sheet3.pdf
    midterm2024.pdf
  analysis/
    exercises.pdf
    tutorial2.pdf
```

2. Student runs `swotvac-saviour sync`. The tool:
   a. discovers immediate subfolders; each folder name becomes a subject tag (sanitized: trimmed, unicode kept, empty names skipped);
   b. scans PDFs inside each folder recursively;
   c. for each new PDF (sha256 idempotent), splits it with the existing parser and transcribes questions with the cheap model;
   d. stores questions, tags, and progress in the existing SQLite schema.
3. Student runs `swotvac-saviour serve` (existing command) and practices.

Folder sync

New `sync` command: `swotvac-saviour sync [--dir .] [--db swotvac.db]`.

- Discovery: immediate subfolders of the target directory. Folder name is the tag. Files are PDFs only (magic-byte gate already exists).
- Idempotency: same sha256 never re-imports. Re-running after adding PDFs imports only the new ones.

- Tag continuity: existing tag rows are matched by name; renaming a folder is a new tag unless explicitly re-keyed via `tag rename` (add to Tag CLI).
- Deterministic parser runs first (free, offline). Its output seeds the transcription request; the model corrects structure rather than starting from a blank page.

Transcription engine (cheap model)

Primary path per question, via OpenRouter (OpenAI-compatible; requires OPENROUTER_API_KEY env var and one dependency: httpx).

- "cheap" defaults: vision-capable low-cost model for reading question text and math, e.g. google/gemini-2.5-flash `[INFERENCE: verify model/price at build time]`. Verifier pass uses a text model (deepseek-chat or z-ai/glm-5.3-flash).
- Pipeline per question:
  - render the question region from the PDF page (existing page-image code) into a PNG crop at ~150dpi;
  - one model call returns JSON: {label, body, body_latex, hint, confidence};
  - deterministic guard rails (already built): sub-parts must stay inside the parent question, label must match the parsed label, control words must be in the known KaTeX set, braces balanced, no HTML;
  - on guard failure, retry once with the image plus a correction hint; the deterministic parse stays as fallback with a warning flag.
- Cost control: batch same-document questions; hard cap per document (default $0.02, configurable); on cap, remaining questions keep the deterministic transcription with warnings.
- Storage: existing questions.body_latex column holds model output; additive columns `engine` and `confidence` (nullable) record provenance. New DBs get them via an additive migration; old DBs are read-compatible.
- Privacy: PDFs leave the machine to the model provider. Flagged: students must not sync private data without accepting this; fully offline mode keeps the deterministic parser as the engine.

Reuse map

Component	Verdict	Notes
parser.py (labels, sub-parts, hints, styles, boundaries)	REUSE	seeds transcription; still the offline engine
normalise.py (ligatures, MATH_MAP, latex_segments, gating)	REUSE	applied to model output before storage

Component	Verdict	Notes
db.py schema + helpers	REUSE	tags already model subjects; additive engine/confidence columns
importer.py (sha dedupe, magic gate, transactional replace)	ADAPT	tags come from folder names during sync; skip auto-create remains
session.py (shuffle, streak, progress, retry)	REUSE	unchanged
web.py play/UI/KaTeX/stats + templates + static	REUSE	unchanged; upload page becomes optional (directive: add flag)
page-image render (badge popover)	REUSE	also feeds transcription crops
CLI parse/import/tag/documents/serve	REUSE	sync added; tag rename added
58 tests	REUSE	extended, never weakened

Retired from the primary flow: the web upload form (folders replace manual upload). Kept behind a config flag, not deleted, until v2 settles.

Site template for the LLM

The deliverable for "template/tool": a scaffold the model fills with question data, then the tool builds and runs the site.

- Scaffold layout (new phase):

```
site-template/
  questions/       <- model writes questions.json here (one file per doc)
  swotvac.yml      <- site config: title, db path, default tags
  templates/, static/, web.py, session.py, db.py  <- copied from this repo
```

- New command: `swotvac-saviour init-site <outdir> [--questions file|dir]`. init-site copies the web/static/templates/modules into outdir, and if –questions points at questions.json or a directory of them, imports them into outdir/swotvac.db and assigns tags (top-level folder name = tag for directory mode).
- After init: `swotvac-saviour serve --db outdir/swotvac.db` (existing command) runs the site. This is the whole interface the LLM needs: write

question JSON with the documented schema, call init-site, done.

- The model-facing contract is one page in the template: the questions.json schema (label, body, body_latex, hint, section, page, tags), plus the two commands (init-site, serve). Everything else is the model writing valid JSON; the tool owns build and hosting.

Question data contract

The single file an LLM (or student tooling) writes. Top level is a JSON array; one element per question. All field names fixed; unknown fields are ignored on import, missing optional fields default as shown.

```
[
  {
    "label": "Exercise 1.1.",
    "section": "Metrics",
    "page": 5,
    "body": "Let (X,d) be a metric space...",
    "body_latex": "Let (X,d) be a metric space. Show that \\mid d(x,y)...",
    "hint": "The Cauchy-Schwarz inequality can be useful.",
    "tags": ["analysis"]
  }
]
```

- `label` (required): unique-ish within a section; used for idempotency and the source badge.
- `section` (optional, default ""): subsection heading shown on the badge.
- `page` (int, optional, default 0): printed page number for the popover.
- `body` (optional): plain-text form, kept for review screens.
- `body_latex` (required): KaTeX-safe markup; must pass the parser's guard rails (balanced braces, known control words, no raw HTML). init-site runs those checks before import and reports failures instead of importing junk.
- `hint` (optional, default null).
- `tags` (optional, default []): subject names; init-site resolves them against existing tags or creates them explicitly (the only auto-create permitted, because the model is the user's agent here).

File modes: single questions.json, or a directory of them (one file per document / per subject, imported with the folder name as the default tag).

omp session workflow

The student's session is the entry point, not a separate app.

1. Student opens an omp/agent session in the study directory (the folder layout above). The tool installs as a runnable command (`uv run swotvac-saviour` / installed script), so any agent in the session can drive it.

2. The repo ships a short agent-facing instruction file (`omp-skill.md` at the repo root, plain text) that tells the agent: three commands exist (sync, init-site, serve), where the DB lives, and where `--engine` is chosen. The student's own workflow never depends on the agent editing code - the agent's job is typing the same two commands the student could.
3. Meaningful paths resolve relative to the study directory: `sync .` discovers the subject subfolders in place; `serve` points at the same directory's database. No copying of student files happens.

Phases

1. Folder sync: discovery, folder-to-tag mapping, tag rename, sync tests.
2. Transcription engine: httpx OpenRouter client, crop pipeline, guard rails, cost cap, engine/confidence columns, fallback behavior.
3. init-site template: scaffold copy, questions.json import, serve wiring, model-facing contract page.
4. Tidy: upload form behind flag, docs, end-to-end tests (folder -> site).

Open decisions

- Model pick and price check for the "cheap" defaults (build-time decision).
- Folder name normalization (case, spaces, non-Latin) for tags.
- Whether PDFs directly in the sync root (no folder) import untagged or are skipped.
- API key handling: env var only, documented.
- Privacy statement wording for the OpenRouter path.

Risks

- Cost runaway on large booklets: mitigated by per-document cap and deterministic fallback.
- Transcription drift (model rewrites the question): mitigated by label match, sub-part integrity, known-control-word and brace checks, plus the source-page popover for verification.
- Folder rename breaks tag continuity: mitigated via tag rename re-key.
- Offline students: deterministic parser remains a working engine.
- Privacy: external transcription is opt-in per sync run (flag –engine llm|pymupdf, default llm if key present, pymupdf otherwise).